Chương 4
TIỂU TRÌNH

NỘI DUNG

1. Tổng quan về tiểu trình
2. Lập trình đa tiểu trình
3. Mô hình đa tiểu trình
4. Thư viện tiểu trình
5. Tiểu trình ẩn
6. Một số vấn đề về tiểu trình
7. Tiểu trình trong một số hệ điều hành

MỤC TIÊU

- ✓ Giới thiệu các thành phần cơ bản của tiểu trình, khác nhau giữa tiểu trình và tiến trình.
- ✓ Mô tả những lợi ích và thách thức của việc thiết kế các ứng dụng đa tiểu trình (đa luồng).
- ✓ Thiết kế các ứng dụng đa luồng bằng các thư viện API: Pthreads, Java và Windows.
- ✓ Khám phá nhiều chiến lược cung cấp tiểu trình ẩn
- ✓ Khảo sát tiểu trình trong HĐH Windows và Linux

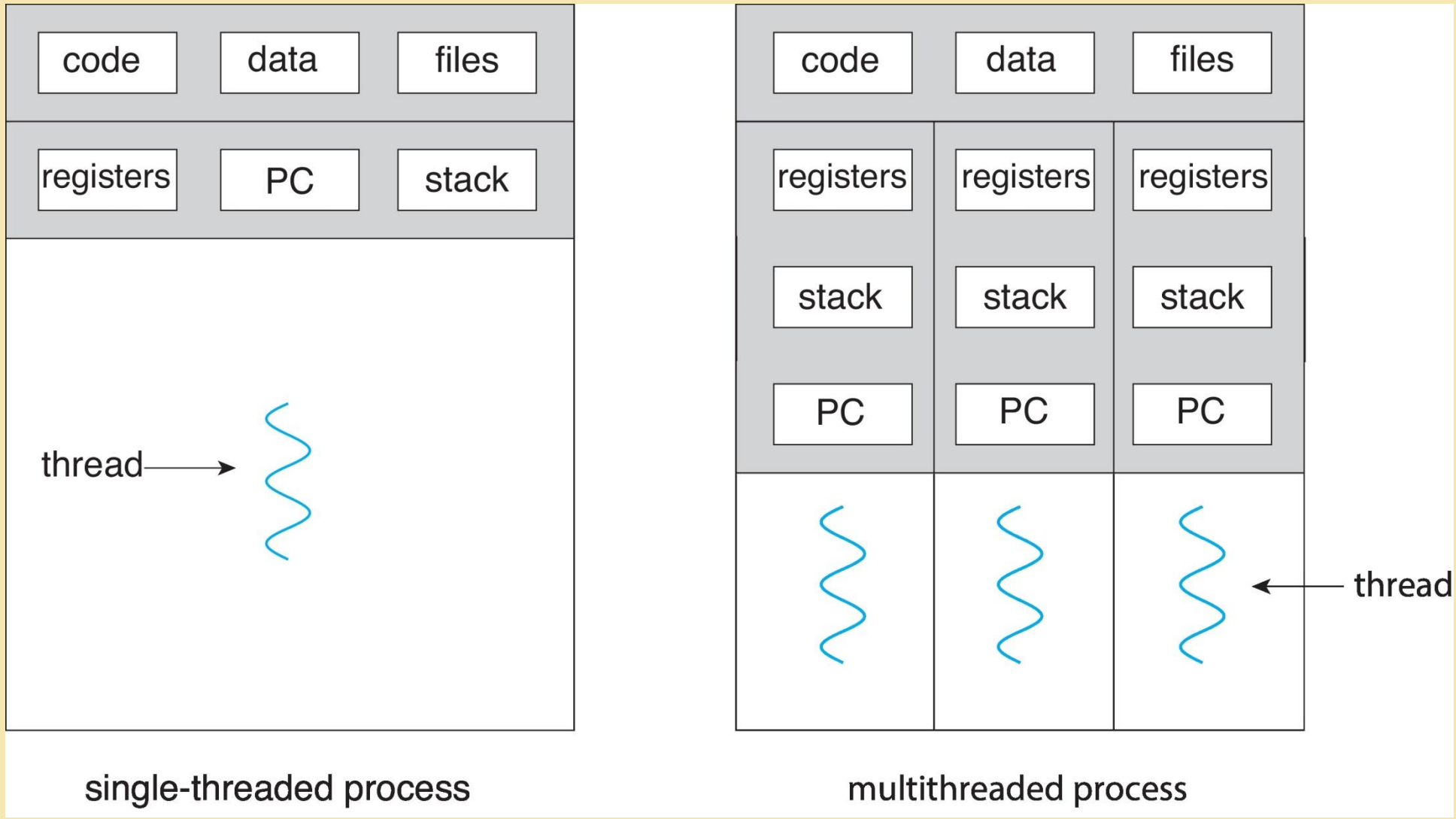
1. Tổng quan về tiểu trình

- Tiểu trình (Thread) (hay còn gọi là tiến trình nhỏ - lightweight process) là một đơn vị cơ bản sử dụng CPU
- Một thực thể trong tiến trình mà hệ điều hành có thể lập lịch để nó thực hiện, không có nó thì các tiến trình của nó không thể thực hiện được
- Bao gồm:
  - Mã định danh ID (identifier)
  - Bộ đếm chương trình (program counter)
  - Tập thanh ghi (register)
  - Ngăn xếp (stack)

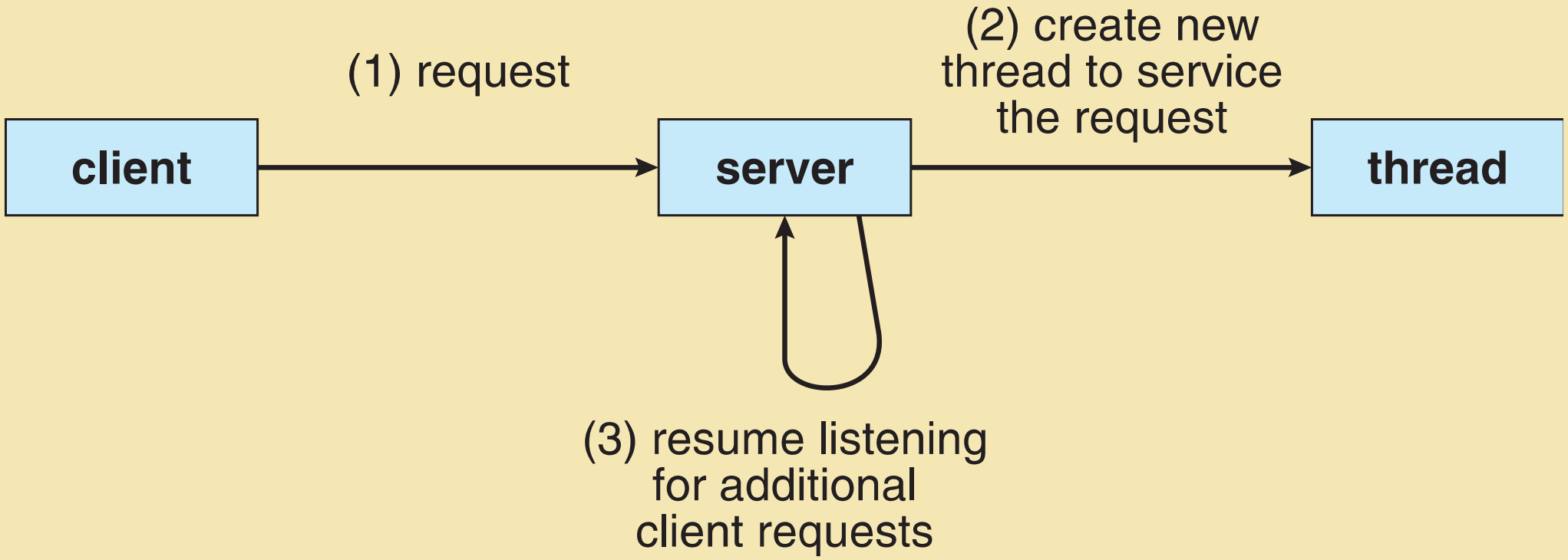
Lý do xuất hiện tiểu trình

- Hầu hết các ứng dụng hiện đại được viết theo hướng đa tiểu trình
- Các tiểu trình chạy bên trong ứng dụng
- Nhiều tác vụ trên ứng dụng có thể được thực hiện bởi nhiều tiểu trình riêng biệt
  - Cập nhật hiển thị
  - Lấy dữ liệu
  - Kiểm tra chính tả
  - Trả lời một yêu cầu trong mạng
- Việc tạo tiến trình thì phức tạp trong khi việc tạo tiểu trình thì đơn giản hơn
- Mã đơn giản, nâng cao hiệu quả
- Nhân HĐH được đa tiểu trình hóa

Tiến trình đơn tiểu trình và tiến trình đa tiểu trình



Kiến trúc web-server đa tiểu trình



Lợi ích của tiểu trình

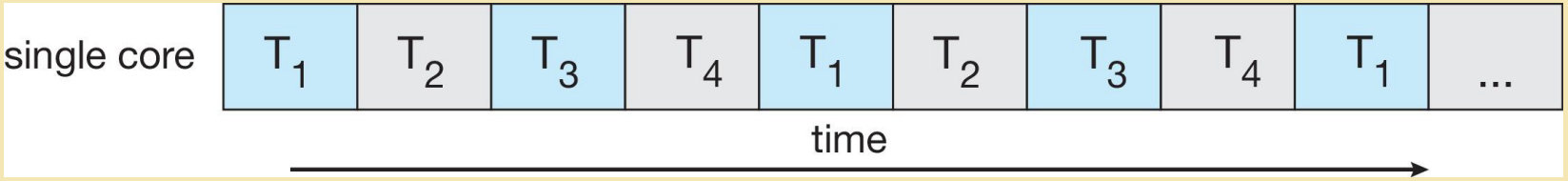
- Khả năng đáp ứng – cho phép tiến trình tiếp tục thực thi nếu một phần nó bị khóa, đặc biệt quan trọng đối với việc thiết kế giao diện người dùng
- Chia sẻ tài nguyên – Các tiểu trình chia sẻ tài nguyên của tiến trình, dễ dàng hơn so với 2 cơ chế bộ nhớ chia sẻ hoặc trao đổi thông điệp
- Tính kinh tế – ít tốn kém hơn so với việc tạo tiến trình, chi phí chuyển ngữ cảnh của tiểu trình thấp hơn so với chuyển ngữ cảnh của tiến trình
- Khả năng mở rộng – tiến trình có thể tận dụng lợi thế của kiến trúc đa xử lý

2. Lập trình đa tiểu trình

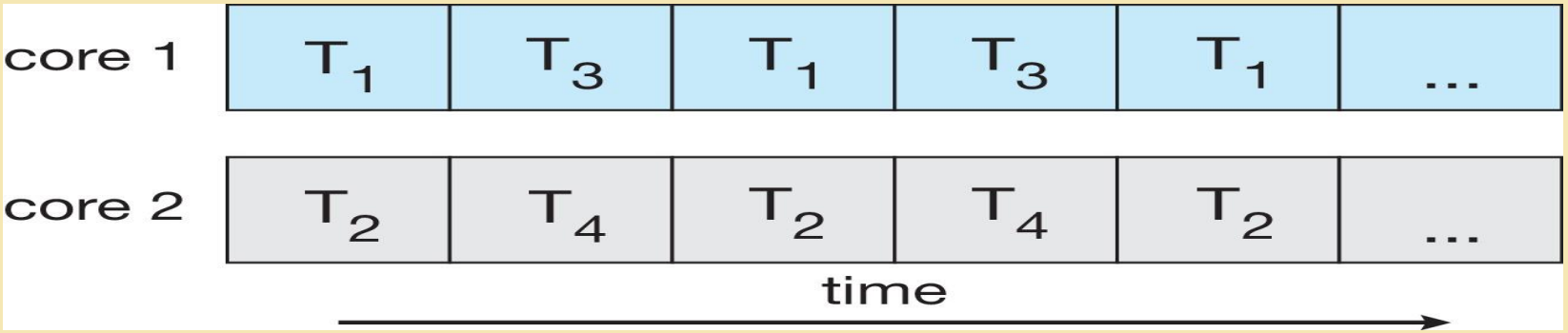
- Hệ thống đa lõi (multicore) hoặc đa xử lý (multiprocessor) đòi hỏi cao đối với các lập trình viên, những thách thức bao gồm:
  - Chia nhỏ hoạt động
  - Cân bằng
  - Tách dữ liệu
  - Phụ thuộc dữ liệu
  - Kiểm tra và gỡ lỗi
- Hệ thống xử lý song song ***(Parallelism)***: có thể thực hiện nhiều tác vụ cùng một lúc
- Hệ thống xử lý đồng thời ***(Concurrency)***: hỗ trợ nhiều hơn một tác vụ được thực hiện theo tiến độ
  - Trên hệ thống single processor/core, bộ định thời cung cấp xử lý đồng thời

Xử lý đồng thời và xử lý song song

- Thực thi đồng thời trên hệ thống đơn lõi (single-core system)

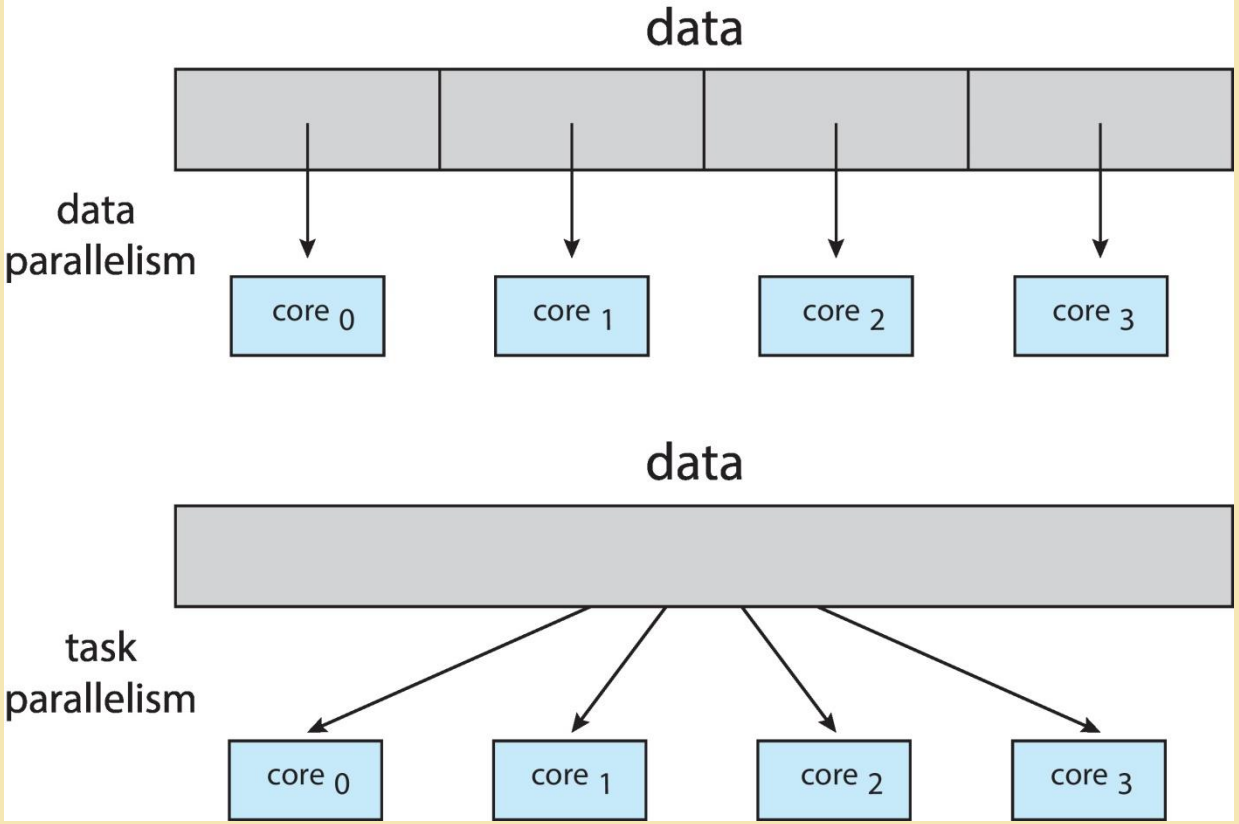


- Thực thi song song trên hệ thống đa lõi (multi-core system)



Lập trình đa tiểu trình

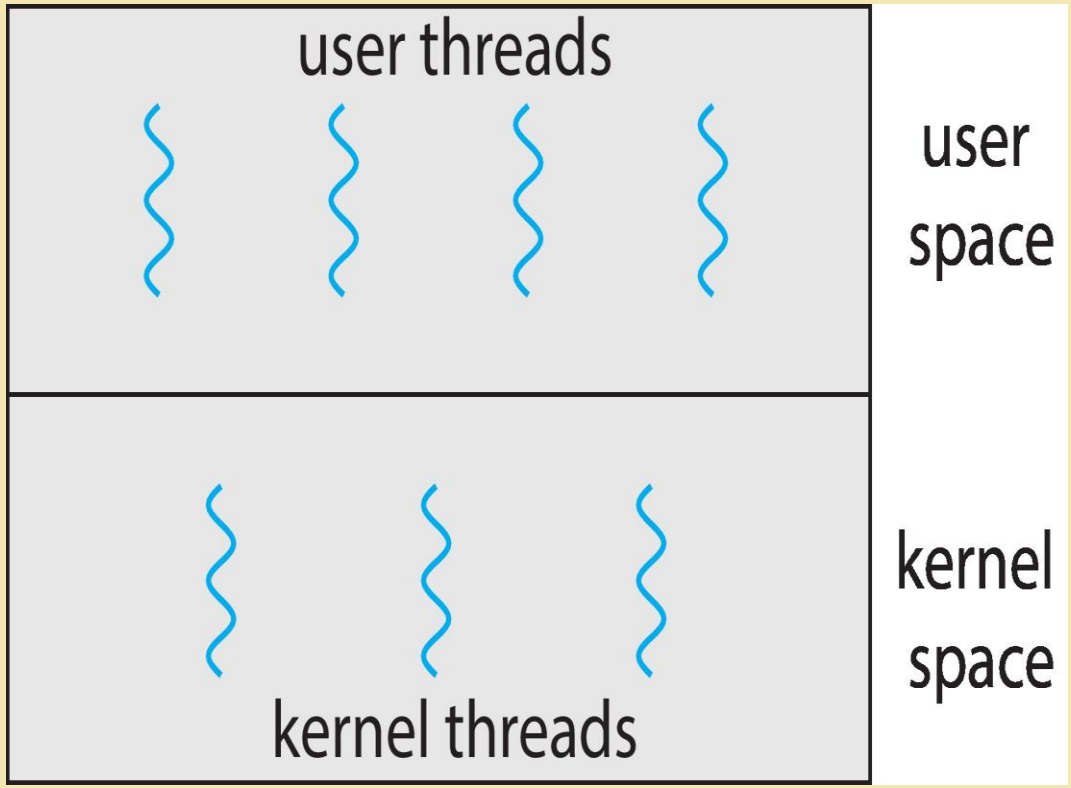
- Các loại xử lý song song
  - Song song dữ liệu (Data parallelism) – phân tán các tập con của dữ liệu ban đầu đến nhiều lõi, các thao tác thực hiện tương tự trên mỗi lõi
  - Song song tác vụ (Task parallelism) – phân phối các tiểu trình đến các lõi, mỗi tiểu trình thực hiện thao tác duy nhất



3. Mô hình đa tiểu trình

Tiểu trình người dùng và tiểu trình nhân

- **Tiểu trình người dùng (User threads)** – quản lý công việc bằng thư viện tiểu trình ở mức người dùng
- Ba thư viện tiểu trình chính:
  - POSIX **Pthreads**
  - Windows threads
  - Java threads
- **Tiểu trình nhân (Kernel threads)** – được hỗ trợ và quản lý trực tiếp bởi HĐH
- Hầu như tất cả các HĐH hiện đại đều hỗ trợ tiểu trình nhân, bao gồm:
  - Windows
  - Linux
  - Mac OS X
  - iOS
  - Android

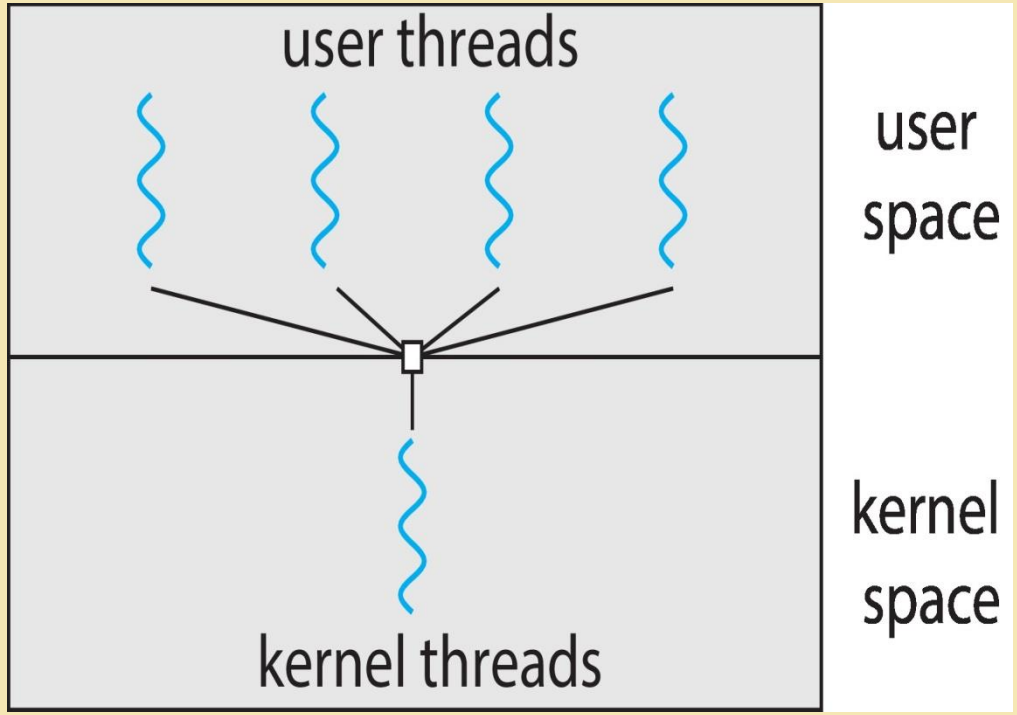


Các mô hình đa tiểu trình

- Many-to-One
- One-to-One
- Many-to-Many

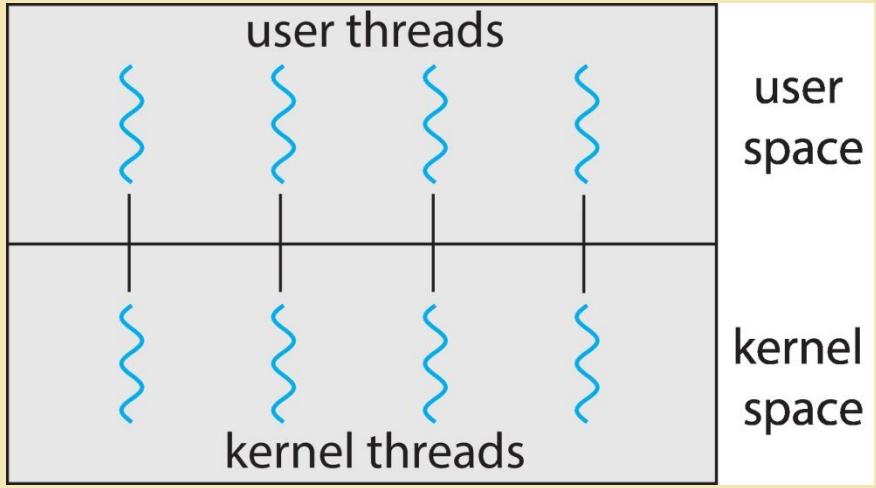
Mô hình Many-to-One

- Nhiều tiểu trình ở mức người dùng được ánh xạ đến tiểu trình nhân duy nhất
- Một tiểu trình bị khóa thì tất cả các tiểu trình cũng bị khóa
- Nhiều tiểu trình có thể không chạy song song trên hệ thống đa lõi bởi vì tại một thời điểm chỉ có một tiểu trình chạy trong chế độ nhân
- Một số hệ thống hiện hành sử dụng mô hình này
  - **Solaris Green Threads**
  - **GNU Portable Threads**



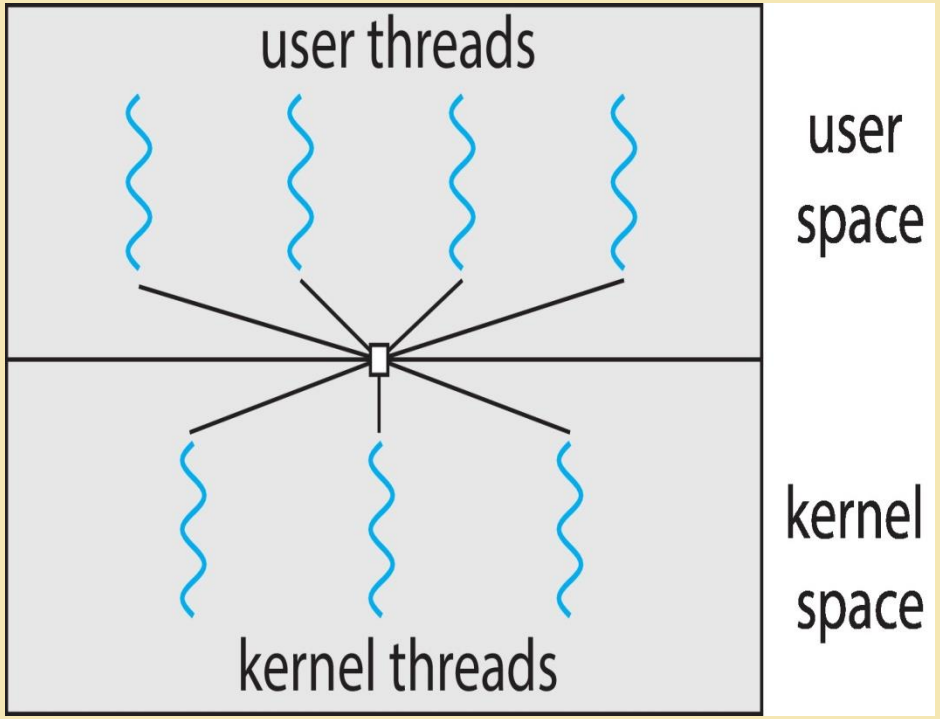
Mô hình One-to-One

- Mỗi tiểu trình mức người dùng ánh xạ đến một tiểu trình mức nhân
- Việc tạo 1 tiểu trình người dùng sẽ tạo một tiểu trình nhân
- Thực thi đồng thời nhiều hơn mô hình Many-to-One
- Số tiểu trình trên tiến trình đôi khi bị hạn chế vì chi phí tính toán
- Các ví dụ
  - Windows
  - Linux



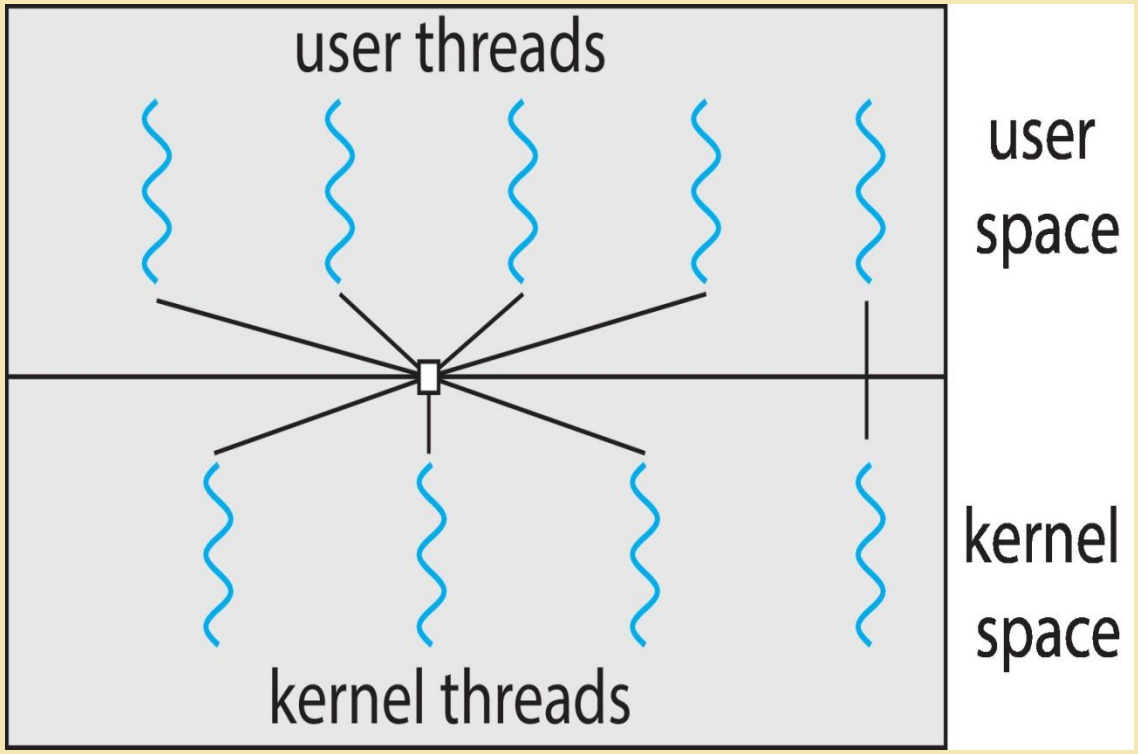
Mô hình Many-to-Many

- Cho phép nhiều tiểu trình mức người dùng được ánh xạ đến nhiều tiểu trình mức nhân
- Cho phép HĐH tạo ra một số lượng tiểu trình nhân đủ để thực hiện
- Windows với gói ThreadFiber hỗ trợ



Mô hình 2 mức

- Giống mô hình many-to-many nhưng cũng cho phép 1 tiểu trình người dùng liên kết đến 1 tiểu trình nhân



4. Thư viện tiểu trình

- Thư viện tiểu trình cung cấp API cho lập trình viên để tạo và quản lý tiểu trình
- Hai cách chính để thực hiện
  - Sử dụng thư viện hoàn toàn ở cấp độ người dùng
  - Sử dụng thư viện tiểu trình ở cấp độ nhân được hỗ trợ bởi HĐH
- Ba thư viện tiểu trình chính:
  - POSIX **Pthreads**
  - Windows threads
  - Java threads

Tạo tiểu trình sử dụng thư viện

- Ví dụ: Chương trình đa tiểu trình thực hiện tính tổng của các số nguyên không âm theo công thức

$$sum = \sum_{i=1}^{N} i$$

- Sử dụng 3 thư viện tiểu trình để viết
  - POSIX **Pthreads**
  - Windows threads
  - Java threads
- 2 chiến lược tạo nhiều tiểu trình
  - asynchronous threading → Tiểu trình cha và tiểu trình con thực thi đồng thời
  - synchronous threading → Tiểu trình cha phải chờ tất cả tiểu trình con kết thúc trước khi thực thi

Thư viện Pthreads

- Có thể được cung cấp ở cả mức người dùng hoặc mức nhân
- API tiêu chuẩn POSIX cho việc tạo và đồng bộ hóa tiểu trình
- Là một bản liệt kê các hoạt động của tiểu trình, không phải là sự thực thi
- Người thiết kế HĐH có thể thực thi bản liệt kê theo bất cứ cách nào họ muốn
- Phổ biến trong HĐH UNIX (Linux, Mac OS X)

Ví dụ chương trình sử dụng Pthreads

```
#include <pthread.h>
#include <stdio.h>

#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
   pthread_t tid; /* the thread identifier */
   pthread_attr_t attr; /* set of thread attributes */

   /* set the default attributes of the thread */
   pthread_attr_init(&attr);
   /* create the thread */
   pthread_create(&tid, &attr, runner, argv[1]);
   /* wait for the thread to exit */
   pthread_join(tid,NULL);

   printf("sum = %d\n",sum);
}
```

Ví dụ chương trình sử dụng Pthreads

```
/* The thread will execute in this function */
void *runner(void *param)
{
   int i, upper = atoi(param);
   sum = 0;

   for (i = 1; i <= upper; i++)
      sum += i;

   pthread_exit(0);
}
```

Mã Pthreads kết nối 10 tiểu trình

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
   pthread_join(workers[i], NULL);
```

Chương trình C sử dụng thư viện Windows Threads

```
#include <windows.h>
#include <stdio.h>
DWORD Sum; /* data is shared by the thread(s) */

/* The thread will execute in this function */
DWORD WINAPI Summation(LPVOID Param)
{
   DWORD Upper = *(DWORD*)Param;
   for (DWORD i = 1; i <= Upper; i++)
      Sum += i;
   return 0;
}
```

Chương trình C sử dụng thư viện Windows Threads

```
int main(int argc, char *argv[])
{
   DWORD ThreadId;
   HANDLE ThreadHandle;
   int Param;

   Param = atoi(argv[1]);
   /* create the thread */
   ThreadHandle = CreateThread(
      NULL, /* default security attributes */
      0, /* default stack size */
      Summation, /* thread function */
      &Param, /* parameter to thread function */
      0, /* default creation flags */
      &ThreadId); /* returns the thread identifier */

    /* now wait for the thread to finish */
   WaitForSingleObject(ThreadHandle,INFINITE);

   /* close the thread handle */
   CloseHandle(ThreadHandle);

   printf("sum = %d\n",Sum);
}
```

Thư viện Java Threads

- Tiểu trình Java được quản lý bởi JVM
- Thường được thực hiện bằng cách sử dụng mô hình tiểu trình được cung cấp bởi hệ điều hành
- Tiểu trình Java có thể được tạo bởi:

```
public interface Runnable
{
    public abstract void run();
}
```

Chương trình đa tiểu trình Java

```
class Sum
{
   private int sum;

   public int getSum() {
     return sum;
   }

   public void setSum(int sum) {
     this.sum = sum;
   }
}

class Summation implements Runnable
{
   private int upper;
   private Sum sumValue;

   public Summation(int upper, Sum sumValue) {
     this.upper = upper;
     this.sumValue = sumValue;
   }

   public void run() {
     int sum = 0;
     for (int i = 0; i <= upper; i++)
        sum += i;
     sumValue.setSum(sum);
   }
}
```

Chương trình đa tiểu trình Java

```
public class Driver
{
  public static void main(String[] args) {
   if (args.length > 0) {
    if (Integer.parseInt(args[0]) < 0)
     System.err.println(args[0] + " must be >= 0.");
    else {
     Sum sumObject = new Sum();
     int upper = Integer.parseInt(args[0]);
     Thread thrd = new Thread(new Summation(upper, sumObject));
     thrd.start();
     try {
        thrd.join();
        System.out.println
                ("The sum of "+upper+" is "+sumObject.getSum());
    } catch (InterruptedException ie) { }
    }
   }
   else
    System.err.println("Usage: Summation <integer value>"); }
}
```

5. Tiểu trình ẩn

- Sự phát triển của các ứng dụng đa tiểu trình với số lượng tiểu trình trong ứng dụng ngày càng tăng, đòi hỏi chương trình chạy chính xác cao → khó khăn nhiều với lập trình viên → giải quyết: tiểu trình ẩn
- Tạo và quản lý tiểu trình được thực hiện bởi trình biên dịch và thư viện run-time (thời gian chạy), chứ không phải là lập trình viên
- Có 3 phương pháp
  - Thread Pools
  - OpenMP
  - Grand Central Dispatch
- Các phương pháp khác bao gồm: Microsoft Threading Building Blocks (TBB), gói **`java.util.concurrent`**

Thread Pools

- Tạo một số tiểu trình trong một vùng lưu trữ, nơi mà các tiểu trình đang chờ đợi công việc
- Ưu điểm
  - Nhanh hơn: đáp ứng một yêu cầu với một tiểu trình hiện có hơn là tạo ra một tiểu trình mới
  - Cho phép số lượng tiểu trình trong ứng dụng được vượt kích thước của vùng lưu trữ
  - Sử dụng các chiến lược khác nhau để chạy tác vụ
- Windows API hỗ trợ thread pools

```
DWORD WINAPI PoolFunction(AVOID Param) {
    /*
    * this function runs as a separate thread.
    */
}
```

OpenMP

- OpenMP là một tập các chỉ thị biên dịch như một API cho các chương trình viết bằng C, C ++, hoặc FORTRAN
- Hỗ trợ cho lập trình song song trong môi trường chia sẻ bộ nhớ
- Xác định vùng song song (parallel regions) - khối mã có thể chạy song song

```
#pragma omp parallel
```

Tạo nhiều tiểu trình như có nhiều lõi

```
#pragma omp parallel for
  for(i=0;i<N;i++) {
     c[i] = a[i] + b[i];
}
```

Chạy vòng lặp song song

```
#include <omp.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
  /* sequential code */

  #pragma omp parallel
  {
    printf("I am a parallel region.");
  }

  /* sequential code */

  return 0;
}
```

Grand Central Dispatch

- Công nghệ của Apple cho HĐH Mac OS X và iOS
- Mở rộng cho ngôn ngữ C, C ++ , API, và thư viện run-time
- Cho phép xác định các bộ phận song song
- Quản lý hầu hết các chi tiết của tiểu trình
- Khối lệnh nằm trong "**^{}**" - **`^{ printf("I am a block"); }`**
- Các khối được đặt trong hàng đợi phân phối
  - Gán tiểu trình có sẵn trong vùng lưu trữ thread khi lấy ra từ hàng đợi

Grand Central Dispatch

- Có 2 loại hàng đợi dispatch
  - Serial – các khối được di chuyển theo FIFO
  - Concurrent – được di chuyển theo FIFO nhưng nhiều khối có thể được di chuyển cùng lúc

```
dispatch_queue_t queue = dispatch_get_global_queue
  (DISPATCH_QUEUE_PRIORITY_DEFAULT, 0);

dispatch_async(queue, ^{ printf("I am a block."); });
```

7. Tiểu trình trong một số hệ điều hành

- Tiểu trình trong Windows
- Tiểu trình trong Linux

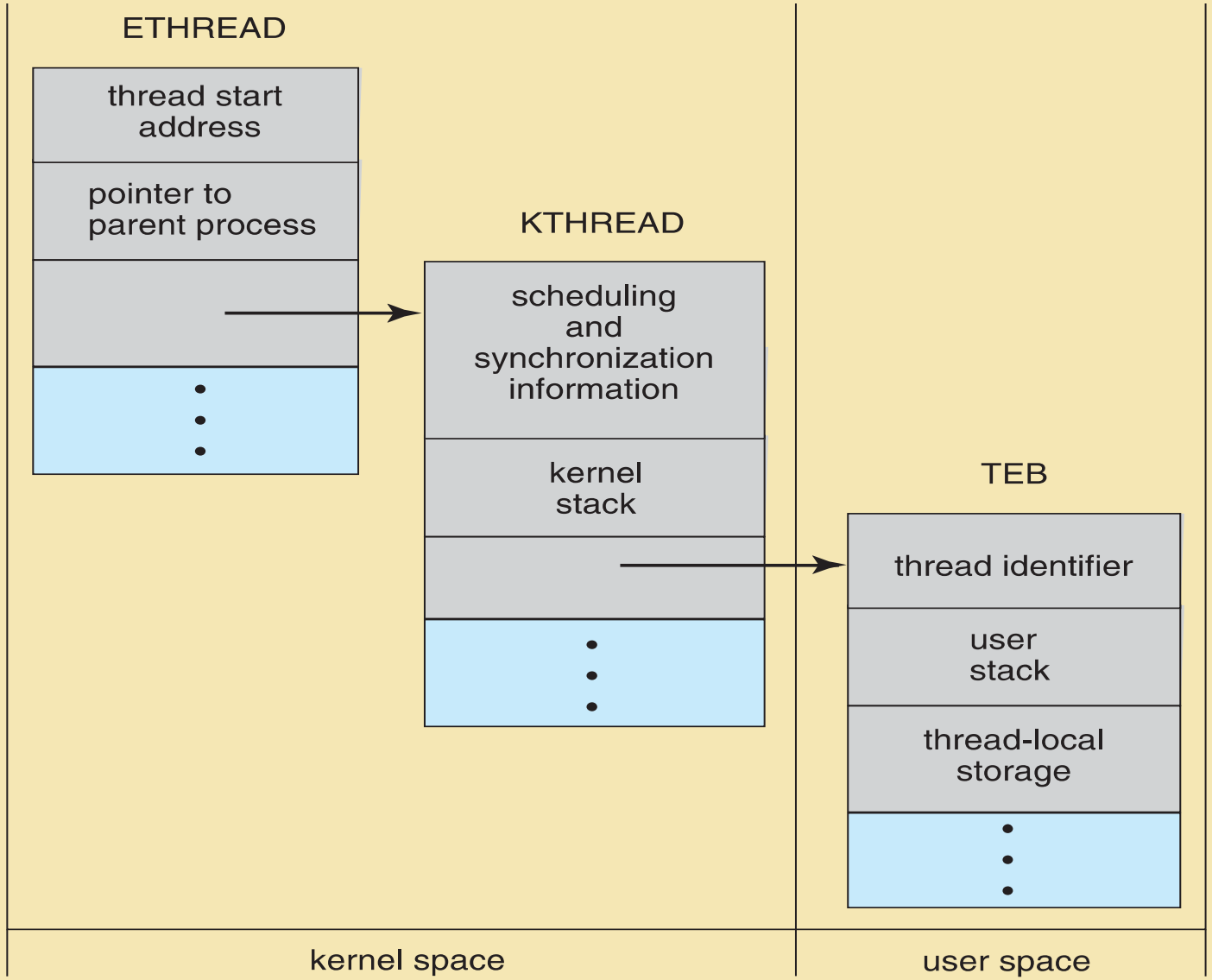
Tiểu trình Windows

- Windows thực hiện Windows API - API chính cho HĐH Windows, gồm Win 98, Win NT, Win 2000, Win XP và Win 7
- Thực hiện một mô hình one-to-one, cấp độ nhân
- Mỗi tiểu trình chứa
  - Mã định danh tiểu trình (thread id)
  - Tập thanh ghi tương ứng trạng thái bộ xử lý
  - Tách ngăn xếp người dùng và ngăn xếp kernel khi tiểu trình chạy trong chế độ người dùng hoặc chế độ hạt nhân
  - Vùng lưu trữ dữ liệu riêng được sử dụng bởi các thư viện run-time và các thư viện liên kết động (dynamic link libraries - DLLs)
- Tập thanh ghi, ngăn xếp, và vùng lưu trữ riêng được biết đến như là ngữ cảnh của thread

Tiểu trình Windows

- Cấu trúc dữ liệu chính của tiểu trình gồm
  - ETHREAD (executive thread block) – bao gồm con trỏ trỏ tới tiến trình mà tiểu trình thuộc về, và một con trỏ trỏ tới KTHREAD. Tồn tại trong không gian nhân
  - KTHREAD (kernel thread block) – chứa thông tin định thời và đồng bộ, một ngăn xếp trong chế độ nhân, con trỏ trỏ tới TEB. Tồn tại trong không gian nhân
  - TEB (thread environment block) – mã định danh tiểu trình, ngăn xếp trong chế độ người dùng, mảng lưu trữ cục bộ. Tồn tại trong không gian người dùng

Cấu trúc dữ liệu tiểu trình Windows



Tiểu trình Linux

- Linux sử dụng tác vụ (Tasks) thay vì tiểu trình (Threads)
- Việc tạo tiểu trình được thực hiện thông qua lời gọi hệ thống ***clone()***
- Clone() cho phép tác vụ con chia sẻ không gian địa chỉ của tác vụ cha (process)
  - Các cờ (Flags) điều khiển

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

Thắc mắc & Giải đáp